

CARRIER ASSURE

Take-Home Technical Assessment

Fullstack Engineer – Miniature Carrier Scoring Service

Assessment Details

Role: Fullstack Engineer

Time Limit: 48 hours from receipt

Estimated Effort: 3–5 hours (AI-assisted workflow expected)

AI Tools: Encouraged – see Section 1.6 for requirements

Submission: GitHub repository link repo

Section 1 – Candidate Instructions

1.1 Overview

At Carrier Assure, we build data-intensive systems that score freight carriers based on compliance and safety data. Our engineering team is currently refactoring performance-heavy data parsers using hash-based change detection.

This assessment evaluates your ability to design, build, and ship a miniature version of this system end-to-end. We value pragmatic engineering decisions, clean architecture, and production-readiness over cleverness or over-engineering.

1.2 The Challenge

Build a Miniature Carrier Scoring Service that ingests a mock Carrier Compliance File (CCF), detects changes efficiently using hash-based comparison, computes a carrier safety score, and displays the results on a simple React dashboard with analytics view. The entire system must run inside Docker containers.

1.3 Objectives

1. Build a backend API (Node.js/TypeScript or Python) that ingests and parses a CCF file (JSON format provided below).
2. Implement hash-based change detection so that re-uploading an identical or partially changed file only re-processes modified carrier records.
3. Compute and persist a composite safety score per carrier based on the fields in the compliance file.
4. Build a React frontend that displays carrier scores.
5. Dockerize the full stack: API service, database, and frontend.
6. Provide a CI/CD pipeline definition (GitHub Actions preferred), unit tests, and a comprehensive README.

1.4 Constraints

- You may use Node.js/TypeScript (preferred) or Python for the backend
- The database must be MongoDB.
- Frontend must be React ([Next.js](#)).
- All services must be defined in a single docker-compose.yml and startable with one command.
- Do not use any external paid services or APIs. All data is mocked/local.

1.5 Definition of Done

Your submission is considered complete when all of the following are true:

- Running **docker-compose up --build** starts the entire system with zero manual steps.
- Uploading a sample CCF file via the API returns carrier scores.
- Re-uploading the same file triggers no re-computation (hash match).
- Modifying one carrier record and re-uploading only re-processes that record.
- The React dashboard loads and displays carrier scores.
- Unit tests pass with npm test or pytest (minimum 70% coverage on scoring logic).
- A .github/workflows/ci.yml file defines at least lint, test, and build stages.
- The README explains architecture decisions, trade-offs, and how to run everything.
- All AI-assisted code is annotated per the AI Usage Policy (Section 1.6).

1.6 AI Usage Policy

At Carrier Assure, AI tools (ChatGPT, Claude, Copilot, Cursor, etc.) are a normal part of our engineering workflow. We actively encourage their use, and you are welcome (even expected) to use AI assistants during this assessment. What we care about is not whether you used AI, but how well you directed it, understood its output, and integrated the result into a cohesive, production-quality codebase.

Mandatory Annotation Requirements

Every piece of code that was substantially generated or heavily influenced by an AI tool must be annotated with a comment block directly above the relevant code. The comment must include the tool used, the prompt (or a summary of the prompt), and a brief note on any modifications you made. This applies to backend code, frontend components, configuration files, CI/CD pipelines, tests, and Docker configuration alike.

Comment Format

Use the following comment block format consistently throughout your codebase:

```
// --- AI-ASSISTED ---
// Tool: Claude 4.5 Sonnet (or ChatGPT-5, Copilot, etc.)
// Prompt: "Generate a SHA-256 hashing function that takes
//   a carrier record object, sorts keys alphabetically,
//   and returns a hex digest"
// Modifications: Added error handling for null fields,
//   changed from MD5 to SHA-256, added TypeScript types.
// --- END AI-ASSISTED ---
```

Section 2 – Technical Specifications

2.1 Carrier Compliance File (CCF) Schema

The input file is a JSON array of carrier records. Each carrier object follows this schema:

Field	Type	Required	Description
carrier_id	string	Yes	Unique identifier (e.g., MC number)
dot_number	string	Yes	USDOT number
legal_name	string	Yes	Legal business name of the carrier
safety_rating	enum	Yes	USDOT safety rating: "Satisfactory", "Conditional", "Unsatisfactory"
out_of_service_pct	number	Yes	Vehicle OOS percentage (0–100)
crash_total	integer	Yes	Total crashes in the last 24 months
driver_oos_pct	number	Yes	Driver OOS inspection percentage (0–100)
insurance_on_file	boolean	Yes	Whether valid insurance is on record
authority_status	enum	Yes	"Active", "Inactive", "Revoked"
last_inspection_date	string	Yes	ISO 8601 date of last roadside inspection
fleet_size	integer	No	Number of power units (default: 1)

```
{
  "carrier_id": "MC-123456",
  "dot_number": "1234567",
  "legal_name": "Reliable Freight LLC",
  "safety_rating": "Satisfactory",
  "out_of_service_pct": 12.5,
  "crash_total": 2,
  "driver_oos_pct": 5.3,
  "insurance_on_file": true,
  "authority_status": "Active",
  "last_inspection_date": "2025-11-15",
  "fleet_size": 45
}
```

2.2 Scoring Algorithm

Compute a composite score from 0 to 100 for each carrier. The algorithm should weigh the following factors. You are free to choose the exact formula, but document your reasoning in the README.

Factor	Weight	Scoring Guidance
safety_rating	25%	Satisfactory = full marks, Conditional = 50%, Unsatisfactory = 0%
out_of_service_pct	20%	Lower is better. 0% = full marks, 100% = zero.
crash_total	20%	0 crashes = full marks. Scale inversely; cap at reasonable max (e.g., 10).
driver_oos_pct	15%	Lower is better. Same inverse scale as vehicle OOS.
insurance_on_file	10%	true = full marks, false = 0.
authority_status	10%	Active = full marks, Inactive = 50%, Revoked = 0%.

2.3 Hash-Based Change Detection

When a CCF file is uploaded, each carrier record must be individually hashed (SHA-256 recommended on the serialized canonical JSON of the record). On subsequent uploads, compare the incoming hash against the stored hash for each carrier_id. Only records whose hash has changed should be re-processed and re-scored. This simulates the performance optimization pattern used in our production CCF parsers.

Why This Matters

In production, Carrier Assure processes files containing thousands of carrier records. Re-scoring unchanged records wastes compute. Your implementation should demonstrate awareness of this pattern. We will test with identical, partially modified, and fully new files.

2.4 API Requirements

Implement the following REST endpoints:

Method	Endpoint	Description
POST	/api/ccf/upload	Accept a CCF JSON file. Parse, hash, detect changes, score changed carriers, persist results. Return a summary: total records, unchanged count, updated count, and new count.
GET	/api/carriers	Return all carriers with their current scores, sorted by score descending. Support optional query params: ?limit=N and ?min_score=N.
GET	/api/carriers/:id	Return a single carrier detail including score breakdown, hash, and last_updated timestamp.
GET	/api/carriers/:id/history	Return score history for a carrier (list of {score, computed_at} entries). Needed for the Advanced Analytics panel.
GET	/api/health	Simple health check returning { "status": "ok", "uptime": ... }.

2.5 Frontend Requirements

Carrier Dashboard

- Display a table/list of all carriers with columns: Legal Name, DOT Number, Score (with color coding: green > 70, yellow 40–70, red < 40), Authority Status.
- Clicking a carrier row navigates to or expands a detail view showing the full score breakdown.
- Include a file upload component that allows the user to submit a CCF file and see a toast/notification with the processing summary.

2.6 Docker & Infrastructure

Required docker-compose.yml Services

1. app – The backend API service (Node.js or Python).
2. db – MongoDB. Must include a health check and data volume.
3. frontend – The React app, served via Nginx or the dev server.

Expectations: Running **docker-compose up --build** from a clean clone should bring the system to a fully functional state. Seed data and migrations (if any) should run automatically on first start.

2.7 CI/CD Pipeline

Provide a .github/workflows/ci.yml (or equivalent) that defines the following stages:

1. Lint – Run ESLint/Prettier (or flake8/black for Python).
2. Test – Run unit tests and report coverage.
3. Build – Build Docker images to verify they compile.

Bonus: Add an integration test stage that spins up docker-compose and runs a smoke test against the /api/health endpoint.

2.8 Testing Requirements

- Minimum 70% code coverage on the scoring algorithm module.
- Unit tests for hash computation and change detection logic.
- At least one test verifying that re-uploading an identical file results in zero re-computations.
- Frontend: at least one component test for the dashboard